Retrieval-Augmented Generation (RAG) → Perplexity AI

traditional LLMs → trained on training data

→ improve the LLM output

1. up to date
2. accuracy ↑
3. fact based ↑

1. Hallucination – making things up
2. Not upto date

Information Retrieval

RAG

Architecture

RAG architecture

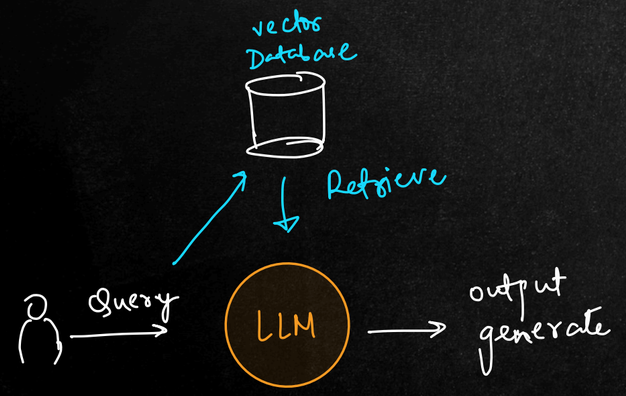
Data Ingestion → Embeddings → store in vector DB

Query → Query Processing

Retrieval

Augmentation

LLM → generation



SQL - traditional DB based

name = "Harry Potter"

vector DB

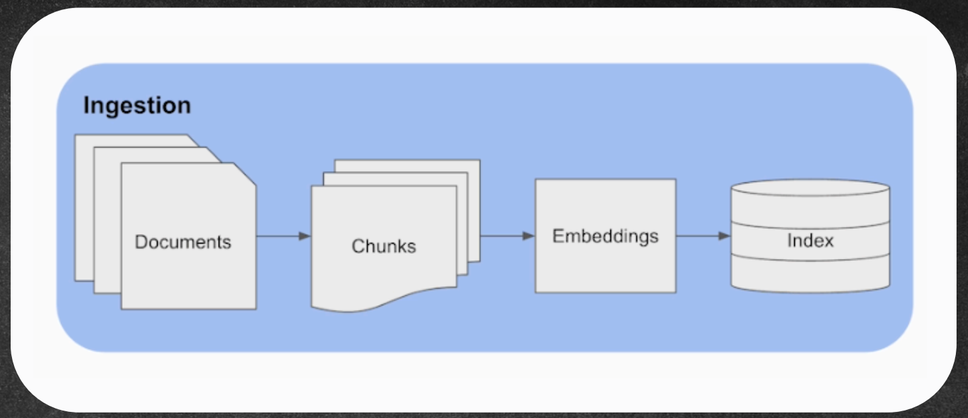
→ wizards, children fantasy

RAG

Data Ingestion Pipeline

context size

LLM



text → nums

vector embeddings

① Data Ingestion → ② Embedding → ③ Vector DB → Knowledge Base

pdf
html
db
excel
⋮

⇓

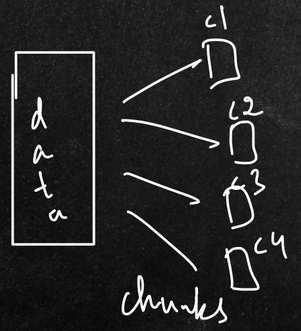
process/clean

⇓

chunks

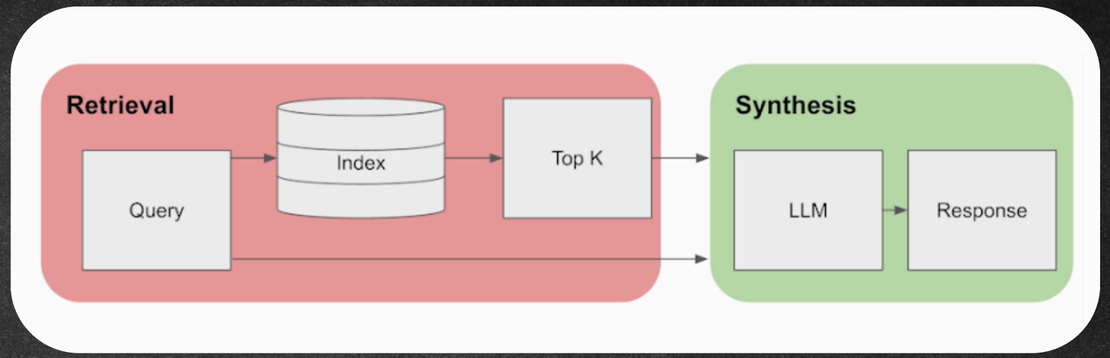
- OpenAI embedding models
- sentence transformer

Pinecone
FAISS
chromadb
⋮



RAG

Retrieval Pipeline



- Nearest neighbour search
- Cosine Similarity (vectors)

cosine dist ↓

cosine similarity = 1 − cosine dist

semantic / similarity search

(5) Retrieval

vector DB

(4) Query Processing

Query Embeddings

Query

(6) Augmentation

context

top K similar results

+ prompt

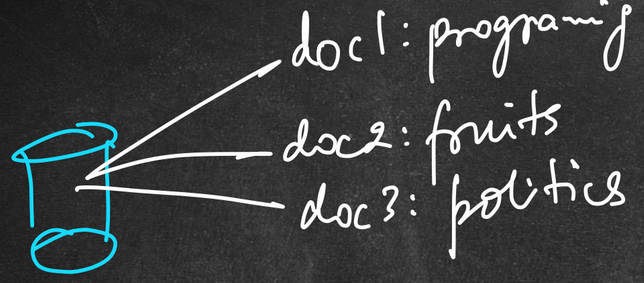
LLM

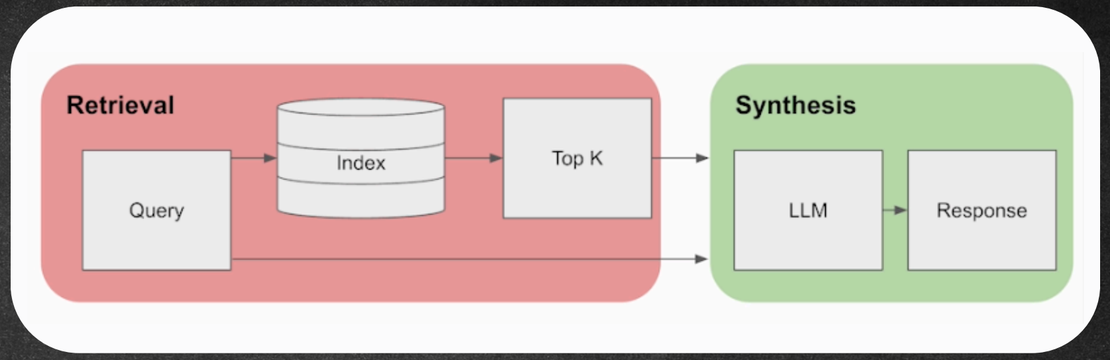
o/p generate

(7) Generation

RAG

Retrieval Pipeline



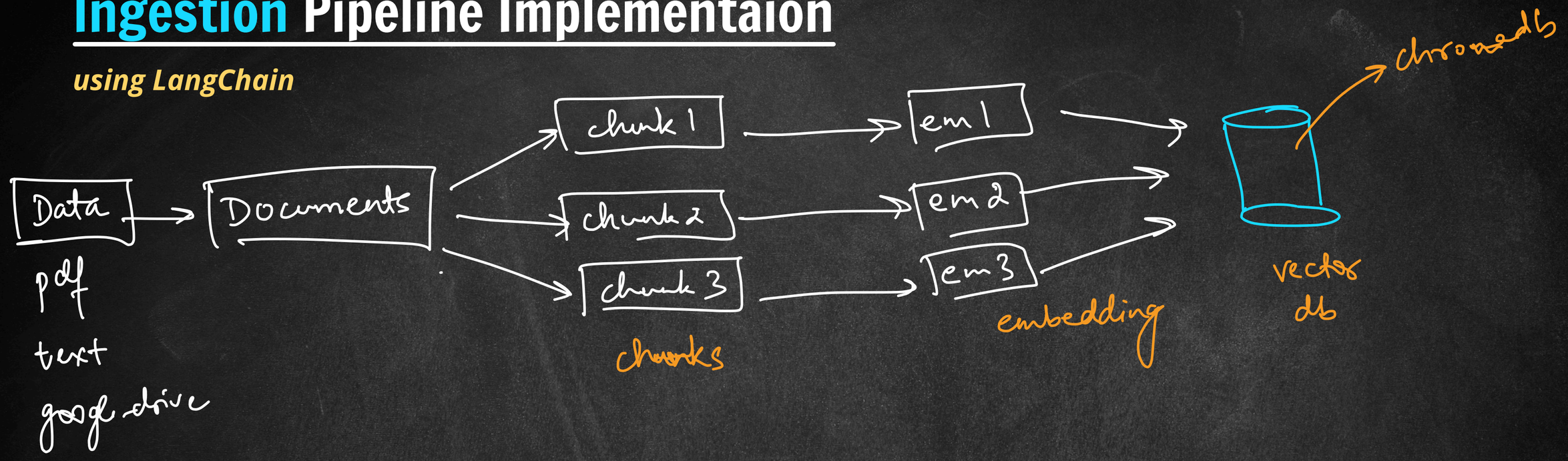


Query Processing → Retrival → Augmentation → Generation

→ Python

Ingestion Pipeline Implementaion

using LangChain



① Document → object

{ ① text/ page-content

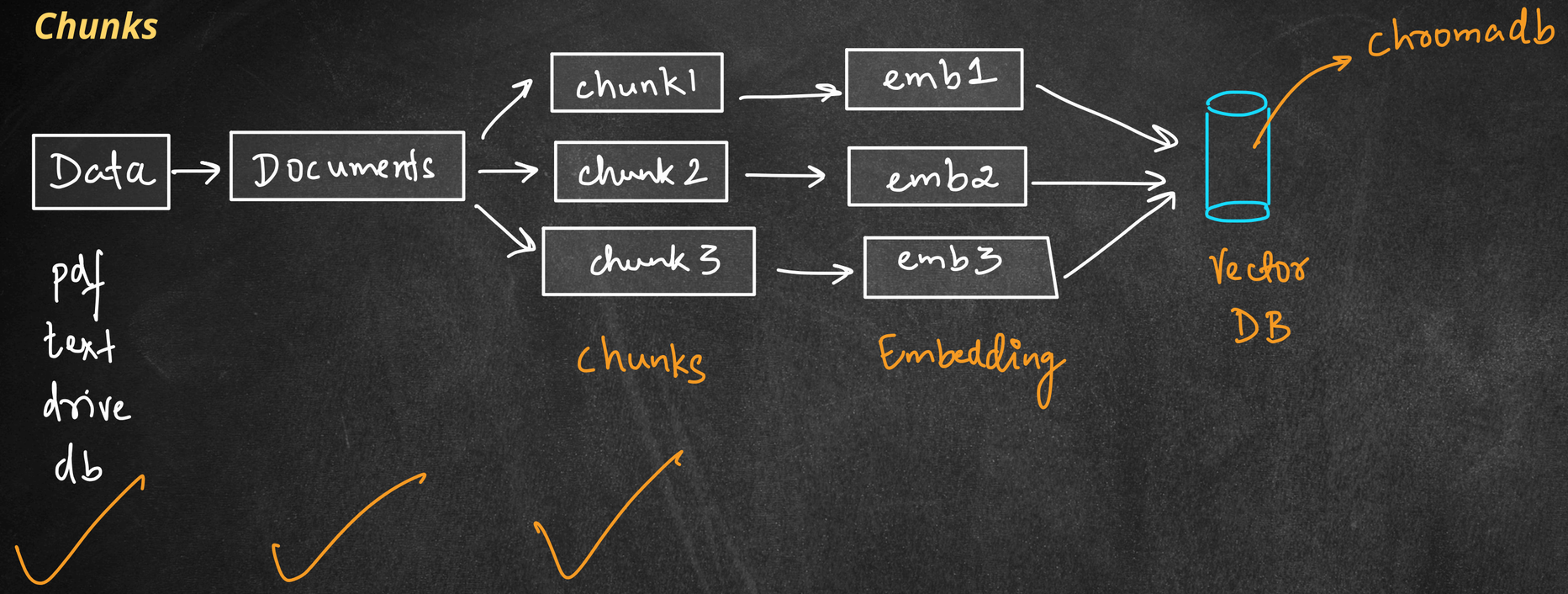
② metadata

}

② Document Loaders

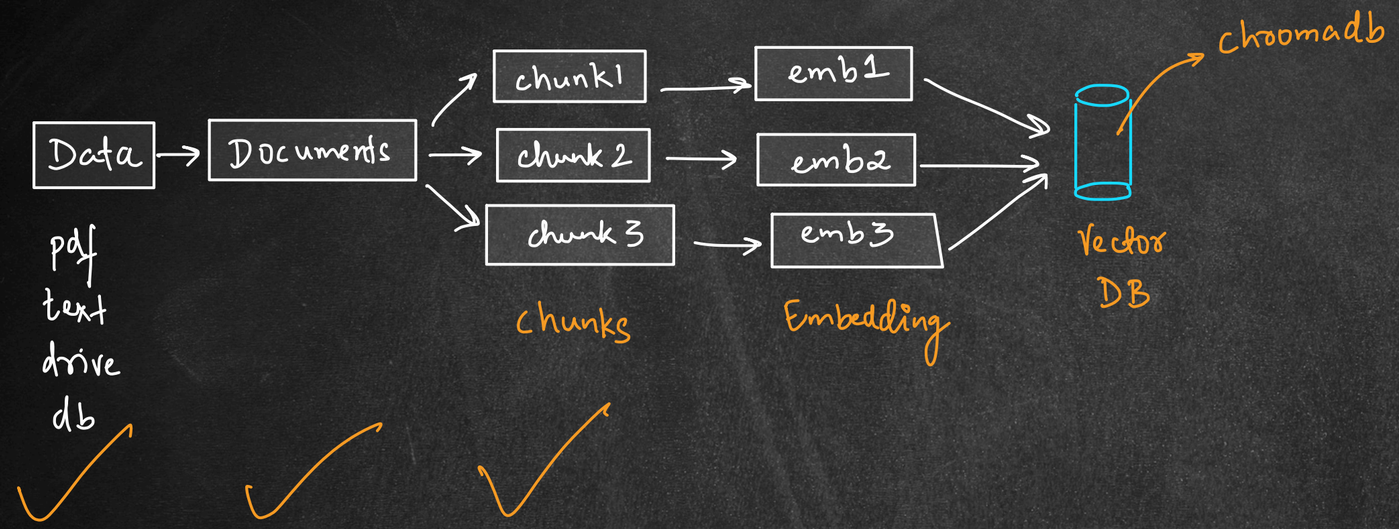
Ingestion Pipeline Implementaion

Chunks



Ingestion Pipeline Implementaion

Embedding



Embedding manager

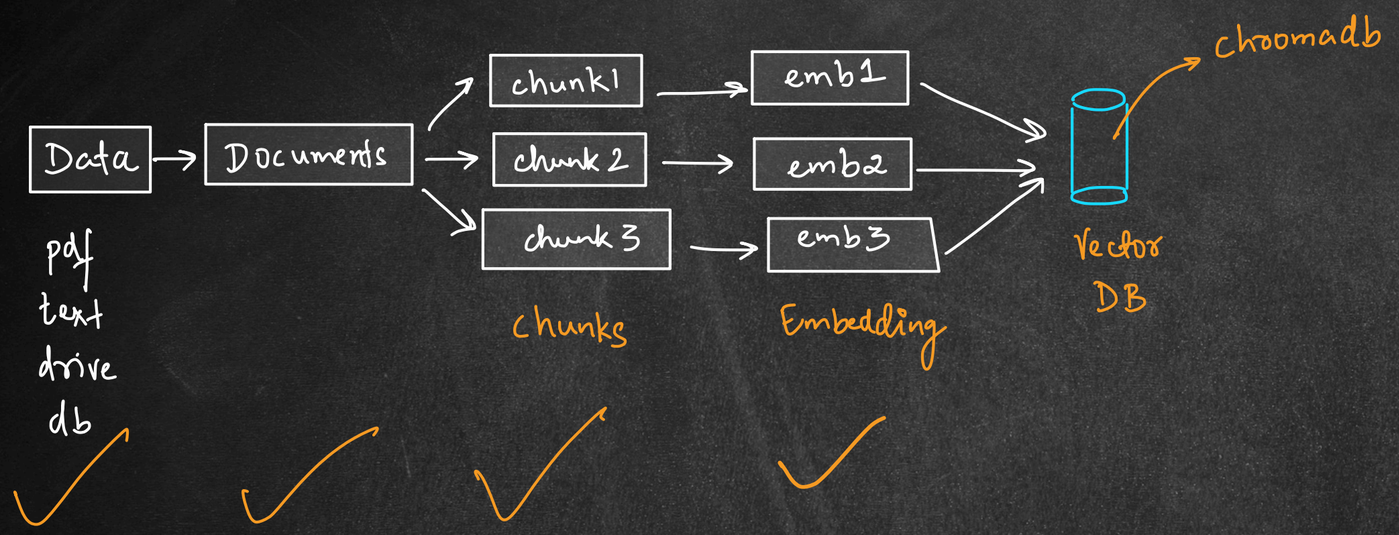
obj

code reusability ↑

Query Embedding

Ingestion Pipeline Implementaion

Vector Store

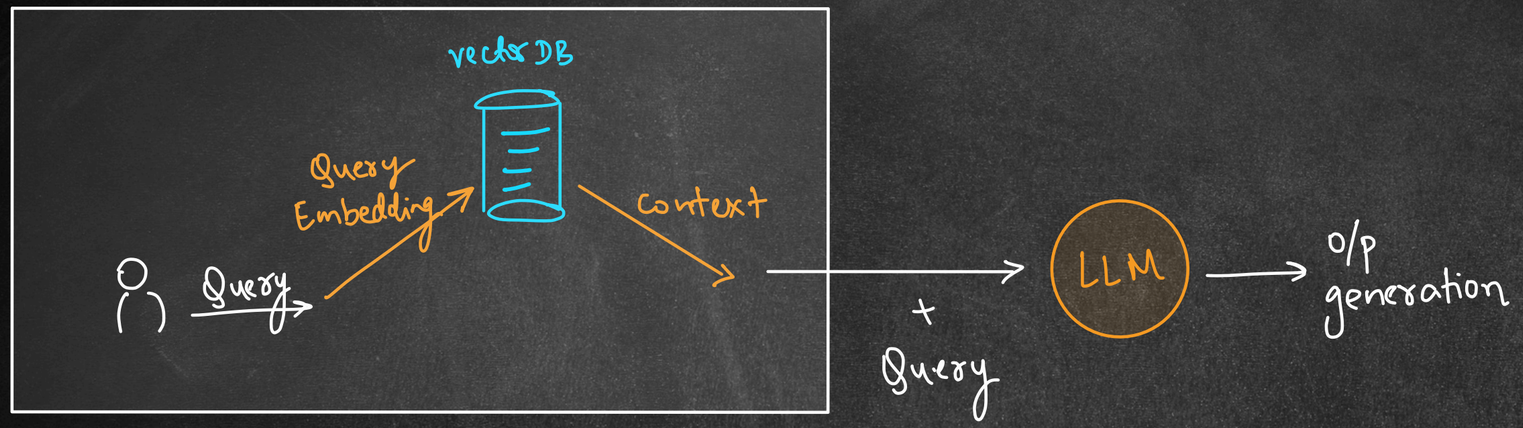


→ Vector Store Manager

1. initialize vector store – constructor
2. VS_initialize → path, collection
3. store docs → VS

Retrieval Pipeline Implementaion

Retriever



RAG Retriever

return a list of results (context)